

## コマ 11: 文字列リテラル + printf

### 今日のゴール

文字列リテラルを `.data` セクションに出力し、`printf` 呼び出しを動かす。

第 10 回までに、ポインタ、配列、関数呼び出し、型サイズを扱えるようになった。この回では、文字列リテラルをメモリ上に配置し、その先頭アドレスを `char *` として使う。

```
printf("x=%d\n", 42);
```

上のコードでは、`"x=%d\n"` をプログラム中のデータ領域に置き、そのアドレスを第 1 引数として `printf` に渡す。

## 1 この回で扱う範囲

対象にする機能は次の通り。

| 種類       | 例                             |
|----------|-------------------------------|
| 文字列リテラル  | <code>"hello"</code>          |
| 外部関数呼び出し | <code>printf("hi\n")</code>   |
| 標準出力テスト  | <code>tests/foo.stdout</code> |

この回では、`printf` 自体は実装しない。コンパイル後のアセンブリを `riscv64-linux-gnu-gcc -static` でリンクすると、`libc` の `printf` が使われる。

## 2 AST を確認する

まず、`printf` と文字列リテラルを含むプログラムがどのような AST になるか確認する。

```
python3 scaffold/parse_viewer.py sessions/11_strings_printf/tests/printf_number.c
```

このプログラムの内容は次の通り。

```
int printf(char *fmt, ...);

int main() {
    printf("x=%d\n", 42);
    return 0;
}
```

実行すると、次のような S 式が表示される。

```
(program
  (funcproto "printf" :type int
    (params (param "fmt" :type (ptr char))))
  (funcdef "main" :type int (params)
    (block
      (exprstmt
        (call "printf"
          (args (str "x=%d\n") (num 42)))))
      (return (num 0)))))
```

新しく重要になるノードは `(str "...")` である。これは `ND_STR` として表され、`node.sval` に文字列の内容が入っている。

### 3 文字列リテラルはどこに置くか

整数リテラル `42` は、命令中に即値として直接埋め込める。しかし文字列リテラルは複数バイトのデータなので、命令列ではなくデータ領域に置く。

```
.data
.LC1:
.byte 120    # 'x'
.byte 61     # '='
.byte 37     # '%'
.byte 100    # 'd'
.byte 10     # '\n'
.byte 0      # 終端 NUL
```

C の文字列は、最後に `0` が入る NUL 終端文字列である。そのため、文字列本体のバイト列を出

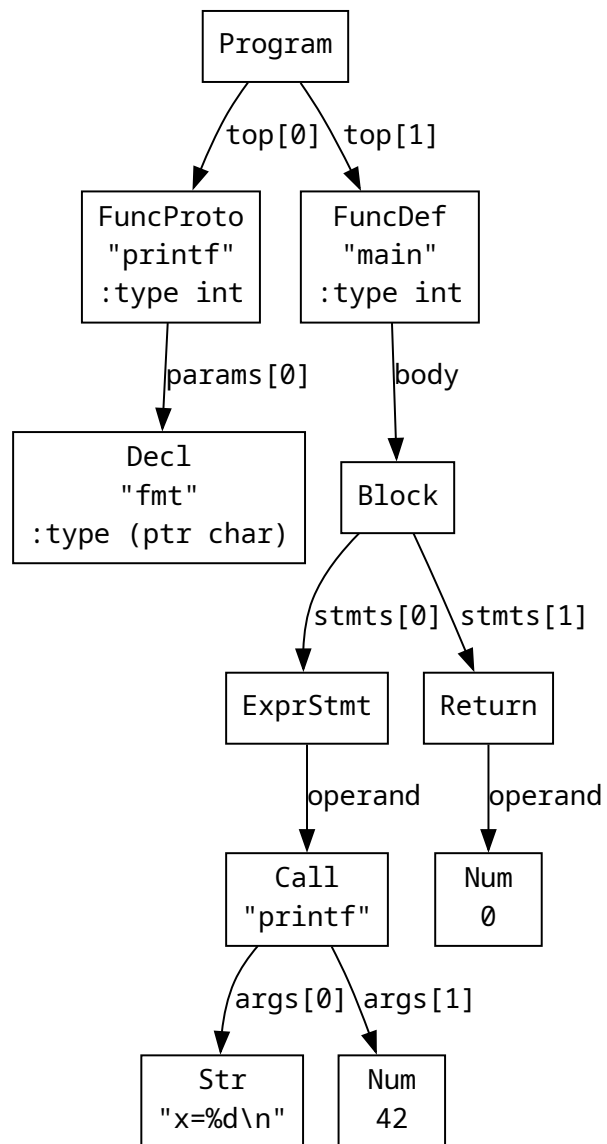
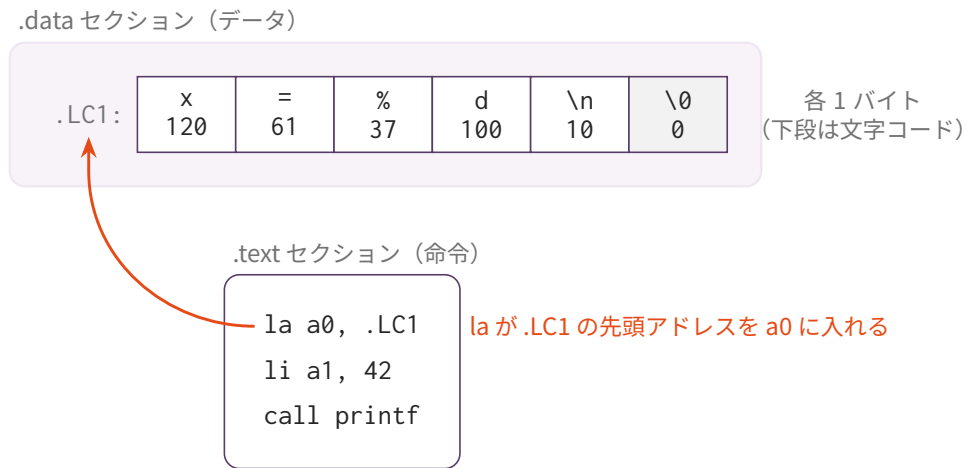


図 1 printf\_number.c の AST

した後に `.byte 0` を必ず出力する。

## 4 .data と .text

アセンブリでは、データと命令をセクションで分ける。



C の文字列は 0 (NUL) で終わる。printf は先頭アドレスから NUL まで読んで出力する

図 2 .data に置いた文字列と、.text からの参照

| セクション | 内容            |
|-------|---------------|
| .data | 文字列リテラルなどのデータ |
| .text | 実行される命令列      |

出力の形は次のようにする。

```
.data
.LC1:
.byte ...
.byte 0

.text
.globl main
main:
...
```

文字列がないプログラムでも .text は必要である。

.text 側の `la a0, .LC1` が、.data に置いた文字列の先頭アドレスを `a0` に入れる。

## 5 文字列ラベル

各文字列リテラルには、一意なラベルを付ける。

たとえば、`"hello"` に `.LC1`、`"x=%d\n"` に `.LC2` のようなラベルを割り当てる。

Codegen クラスは `instance` 変数で文字列ラベルを管理する。

```
# __init__ 内
self._strings: dict[str, str] = {}
self._label_counter = 0
```

同じ文字列が複数回出てきた場合は、同じラベルを再利用してもよい。実装を単純にするため、`node.sval` をキーにした辞書で管理する。

```
def _intern(self, s: str) -> str:
    if s not in self._strings:
        self._label_counter += 1
        self._strings[s] = f".LC{self._label_counter}"
    return self._strings[s]
```

## 6 ND\_STR のコード生成

文字列リテラルを式として評価すると、その文字列の先頭アドレスが得られる。型は `char *` と考える。

```
if node.kind == ND_STR:
    label = self._intern(node.sval)
    self.emit(f" la a0, {label}")
    return
```

`la` は疑似命令で、ラベルのアドレスをレジスタに入れる。この結果、`a0` に文字列先頭アドレスが入る。

## 7 printf 呼び出し

`printf` は普通の関数呼び出しと同じ ABI で呼び出せる。

```
printf("x=%d\n", 42);
```

この呼び出しでは、引数は次のように渡される。

| 引数 | 内容               |
|----|------------------|
| a0 | "x=%d\n" の先頭アドレス |
| a1 | 42               |

第 08 回で作った関数呼び出しのコード生成がそのまま使える (`self.gen_call()` または `ND_CALL` ハンドラメソッド)。新しく必要なのは、`ND_STR` を評価したときに文字列アドレスを `a0` に入れる処理である。

## 8 #include "lib.h"

実際のプログラムでは、`printf` の宣言を自分で書く代わりに `lib.h` を `include` できる。

```
#include "lib.h"

int main() {
    printf("Hello, World!\n");
    return 0;
}
```

`scaffold/lib.h` には次の宣言が用意されている。

```
int printf(char *fmt, ...);
```

`...` は可変長引数を表す。この講義の Parser は外部関数宣言に限って `...` を読み飛ばす。呼び出し側では、通常に関数呼び出しと同じように引数を左から評価すればよい。

## 9 標準出力テスト

これまでのテストは、主に終了コードを `.ans` で確認してきた。第 11 回では標準出力も確認する。

```
tests/printf_hello.c
tests/printf_hello.ans
tests/printf_hello.stdout
```

`.ans` には終了コードを書く。`.stdout` には期待する標準出力をそのまま書く。

たとえば、次のプログラムなら、

```
printf("Hello, World!\n");
return 0;
```

`.ans` は次の通り。

```
0
```

`.stdout` は次の通り。

```
Hello, World!
```

## 10 実装手順

1. スケルトンの `importlib` 継承により第 10 回の `Codegen` クラスを引き継ぐ（あらかじめ書かれている）
2. `self._strings` と `self._intern()` を追加して文字列ラベル管理を作る
3. AST 全体を走査して、出現する文字列を事前に登録する
4. `.data` セクションを出力し、各文字列を `.byte` 列として出す
5. `.text` セクションを出力して、関数本体を出す
6. `self._type_of_expr(node)` で `ND_STR` を `char *` にする
7. `ND_STR` のハンドラメソッドで `la a0, label` を出す
8. `printf_hello.c` と `printf_number.c` の `stdout` テストを通す

## 11 テスト

```
python3 scaffold/test_runner.py sessions/11_strings_printf
```

この回の主要テストは次の通り。

| テスト                           | 内容                           | 期待                         |
|-------------------------------|------------------------------|----------------------------|
| <code>printf_hello.c</code>   | 文字列だけを出力する                   | <code>Hello, World!</code> |
| <code>printf_number.c</code>  | <code>%d</code> に整数を渡す       | <code>x=42</code>          |
| <code>strlen_literal.c</code> | 文字列を <code>strlen</code> に渡す | 終了コード 3                    |